



**UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO
FACULTAD DE INGENIERÍA
COMPILERS. GROUP 05**

PROFFESOR: RENÉ ADRIÁN DAVILA PÉREZ.

Deadline: 28 de Abril del 2026

PARSER AND SDT

Semester 2026-2

TEAM 02

Student names	Account numbers
Contreras del Ángel Diego Adrián	32029883-5
López López José Rodrigo	32009844-6
Medrano Solano Enrique	32015284-1
Perez Rivas Evelyn Samantha	32025818-7
Rodríguez Zamora Joshua	32022959-4

Contents

1 Introduction:	3
2 Background:	3
2.1 Parser:	3
2.2 Applications and Classifications of Syntax Analyzers	3
2.3 Key applications:	4
2.4 Types of parsers:	4
2.5 Development challenges:	5
2.6 The future of parsing:	5
3 Syntax-Directed Translation (SDT):	5
3.1 Attributes	6
4 Development and Body	9
4.1 Formal Grammar Definition:	9
4.2 Syntactic Analysis and AST Generation:	10
4.3 Syntax-Directed Translation (SDT) and Type Checking	11
5 Results	12
5.1 TEST 1:	12
5.2 TEST 2:	13
5.3 TEST 3:	14
5.4 TEST 4:	15
5.5 TEST 5:	16
6 Conclusions:	17

1 Introduction:

- **Problem statement:** The primary challenge of this project involves the transition from simple tokenization to the development of a predictive parser and Syntax Directed Translation scheme. This involves not only decomposing raw input into tokens but also validation that these tokens follow a specific structural logic defined by a Context Free Grammar.
- **Motivation:** The motivation behind this project is to bridge the gap between recognizing symbols and interpreting meaning. By implementing a Parser and SDT, we explore the transition from Finite Automata to Pushdown Automata, gaining a deeper understanding of how modern compilers and interpreters validate code structure.
- **Objectives:** The primary objective is to develop a functional Syntax Directed Translator, implement a parser capable of verifying if a sequence of tokens conforms to a formal Context Free Grammar, apply formal transformations to the grammar to ensure it is suitable for a non backtracking, top down parsing implementation and integrate translation schemes within the grammar rules to perform specific tasks, such as generating an intermediate representation, evaluating arithmetic expressions, or managing a symbol table.

2 Background:

2.1 Parser:

A syntax analyzer, also known as a parser, is a tool used in natural language processing to analyze and understand the syntactic structure of a sentence or text. The goal of a parser is to determine the relationship between words and phrases within a text and their grammatical function in the context of the entire document.

2.2 Applications and Classifications of Syntax Analyzers

Overview of Parsing Techniques:

A parser employs various techniques to analyze text, primarily top-down or bottom-up parsing. These methods rely on the generation of syntax trees (or parse trees) that represent the structural hierarchy of the input. These trees are essential for determining the grammatical function of each element within the context of the complete source text or sentence.

2.3 Key applications:

Parsers are indispensable in several fields:

- Natural Language Processing (NLP): Used in chatbots, virtual assistants, and sentiment analysis.
- Machine Translation: Facilitating the conversion of one human language to another while preserving structural meaning.
- Compiler Development: Translating high-level source code into machine-executable instructions.
- Data Mining and Indexing: Extracting structured information from vast amounts of unstructured text.

2.4 Types of parsers:

The classification of a parser often depends on its processing direction and the complexity of the language it handles:

Type	Description
Top Down Parsers	Start from the highest level grammar rule and work down to individual words or tokens.
Bottom Up Parsers	Begin with individual tokens and combine them into increasingly larger structures until reaching the start symbol.
Shift Reduce Parsers	An efficient bottom-up approach that uses a stack to "shift" tokens and "reduce" them into grammatical structures.
Syntax Tree Parsers	Focused specifically on generating a hierarchical representation of the grammatical roles within the text.
Regular Language Parsers	Designed to process formal languages that follow strict patterns (Regular Expressions), commonly used in lexical analysis.
Natural Language Parsers	Specialized in handling the nuances, Languages, and irregular structures of human speech and writing.

2.5 Development challenges:

Developing robust parsers, particularly for NLP, involves overcoming significant hurdles:

- **Syntactic Ambiguity:** Resolving instances where a single phrase can have multiple valid structural interpretations.
- **Linguistic Variability:** Adapting to diverse dialects, slang, and evolving language styles.
- **Computational Complexity:** Optimizing algorithms to handle large-scale data without sacrificing real-time performance.
- **System Integration:** Ensuring seamless operation with machine learning models and speech recognition APIs.

2.6 The future of parsing:

- **The evolution of parsers is increasingly driven by Artificial Intelligence:**
- **Deep Learning Integration:** Moving beyond rule-based systems to neural parsers that understand context more deeply.
- **Multilingual Capabilities:** Developing universal parsers that maintain high accuracy across different language families.
- **Domain Personalization:** Automated tools that can be fine-tuned for specific industrial or academic vocabularies.

3 Syntax-Directed Translation (SDT):

SDT is a method used in compiler design to convert source code into another form while analyzing its structure. It integrates syntax analysis (parsing) with semantic rules to produce intermediate code, machine code, or optimized instructions. Each grammar rule is linked with semantic actions that define how translation should occur. These actions help in tasks like evaluating expressions, checking types, generating code, and handling errors.

SDT ensures a systematic and structured way of translating programs, allowing information to be processed bottom-up or top-down through the parse tree. This makes translation efficient and accurate, ensuring that every part of the input program is correctly transformed into its executable form.

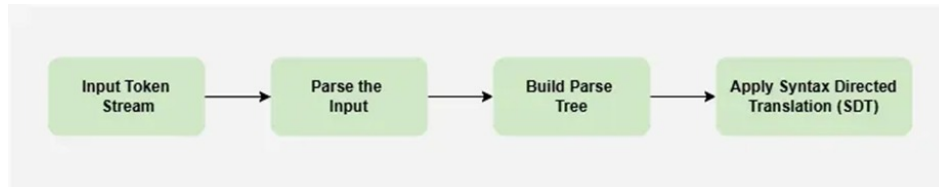


Figura 1: SDT executable form.

Key Elements:

1. Lexical values of nodes (such as variable names or numbers).
2. Constants used in computations.
3. Attributes associated with non-terminals that store intermediate results.

General process:

- First, a parse tree or syntax tree is constructed.
- Then, attribute values are computed by visiting the tree nodes in a specific order.
- In many cases, translation is performed directly during parsing without explicitly building the tree.

3.1 Attributes

An attribute is any quantity associated with a programming construct in a parse tree. Help in carrying semantic information during the compilation process.

Examples of Attributes:

- Dat types of variables
- Line numbers for error handling
- Instruction details for code generation

Types of attributes

Synthesized Attributes:

- Defined by a semantic rule associated with the production at node N in the parse tree.
- Computed only using the attribute values of the children and the node itself.
- Mostly used in bottom-up evaluation.

Inherited Attributes:

- Defined by a semantic rule associated with the parent production of node N.
- Computed using the attribute values of the parent, siblings, and the node itself.
- Used in top-down evaluation.

Attribute Grammars

Special type of grammar used in compiler design to add extra information (attributes) to syntax rules. This helps in semantic analysis, such as type checking, variable classification, and ensuring correctness in programming languages.

Example of an Attribute Grammar:

Production Rule	Semantic Rule	Explanation
$D \rightarrow T L$	$L.in := T.type$	The type defined in T is passed to L.
$T \rightarrow int$	$T.type := integer$	Assigns the data type integer to T.
$T \rightarrow real$	$T.type := real$	Assigns the data type real to T.
$L \rightarrow L , id$	$L.in := L.in \text{ addtype}(id.entry, L.in)$	Passes type information to L and updates the symbol table with the identifier type.
$L \rightarrow id$	$\text{addtype}(id.entry, L.in)$	Assigns the type information to the identifier id and stores it in the symbol table.

Translation Rules:

Production Rule	Translation Action
$E \rightarrow E + T$	{ print('+') }
$E \rightarrow E - T$	{ print('-') }
$E \rightarrow T$	—
$T \rightarrow 0$	{ print(0) }
$T \rightarrow 1$	{ print('1') }
...	...
$T \rightarrow 9$	{ print('9') }

Translation rules can be evaluated using Depth-First Search (DFS) traversal of the parse tree.

When all attributes are synthesized, the attributes of child nodes are computed before the parent node, so evaluation can be done in one traversal. If other types of attributes are present, we may need to determine the proper order of traversal and possibly perform multiple passes on the parse tree. Evaluation is performed in a bottom-up and left-to-right manner to compute translation rules correctly.

Grammar with Semantic Actions

$E \rightarrow E + T$ $E.val = E.val + T.val$ (PR1)

$E \rightarrow T$ $E.val = T.val$ (PR2)

$T \rightarrow T * F$ $T.val = T.val * F.val$ (PR3)

$T \rightarrow F$ $T.val = F.val$ (PR4)

$F \rightarrow INTLIT$ $F.val = INTLIT.lexval$ (PR5)

Each production rule has a semantic action in that defines how values are computed.

E, T, and F are non-terminals (expression components). INTLIT represents an integer literal (actual number). val is an attribute used to store computed values at each step.

4 Development and Body

For the implementation of this project, a hybrid and modular architecture. This means we divided the project into two main parts to make it more organized.

- **Lexical and Syntax Analyzer (Parser and Lexer):** Developed in C, a Top-Down Parsing method, specifically a Recursive Descent Parser, was chosen. The parser processes the input token stream sequentially, validates it against our Context-Free Grammar (CFG), and generates an Abstract Syntax Tree (AST) formatted as a JSON string.
- **Semantic Analyzer and Interface (SDT and GUI):** Developed in Python using the customtkinter library. Python acts as an orchestrator that executes the compiled C script, captures the JSON AST, and performs the Syntax-Directed Translation SDT, evaluation to verify type congruence before displaying the results.



Figura 2: General execution of the code.

4.1 Formal Grammar Definition:

To validate the code written by the user, we created a Context-Free Grammar, you can think of this grammar as the strict “rulebook” of our compiler. Every time the user types a declaration, the parser checks if it follows these exact rules.

If the order of the words or symbols is wrong, the system immediately throws a , grammar is designed to be simple and direct, avoiding ambiguity so the computer always knows what to do.

$$G = \{N, \Sigma, P, S\}$$

$$P : \{Declaration \rightarrow TypeId = Expression;$$

$$Type \rightarrow int | float | string | bool$$

$$Expression \rightarrow number | stringVal | boolVal\}$$

Figura 3: Grammar.

4.2 Syntactic Analysis and AST Generation:

For the syntactic analysis, we implemented a method called Top-Down Parsing, this means the program starts looking at the biggest rule, a full declaration and breaks it down into smaller pieces (the type, the variable name, the equals sign and the value). While reading the code from left to right, the C script builds an Abstract Syntax Tree (AST).

An AST is a very important data structure, it does not just save the text; it organizes the important parts of the code in memory. For example, our AST creates a node that explicitly saves the variable's declared type, its identifier name, and type of the expression assigned to it.

```
// ESTRUCTURA
typedef struct ASTNode {
    char node_type[50];
    char var_type[50];
    char id_name[50];
    char expr_type[50];
    char expr_value[150];
    char semi[10];
} ASTNode;
```

Figura 4: Implementation of the AST Node Structure

```
ASTNode* parse_declaration() {
    ASTNode* node = (ASTNode*)malloc(sizeof(ASTNode));
    strcpy(node->node_type, "VarDecl");

    strcpy(node->var_type, current_token.lexeme);
    eat(T_TYPE);

    strcpy(node->id_name, current_token.lexeme);
    eat(T_ID);

    eat(T_ASSIGN);

    strcpy(node->expr_value, current_token.lexeme);

    if (current_token.type == T_NUMBER) {
        if (strchr(current_token.lexeme, '.') != NULL) {
            strcpy(node->expr_type, "float");
        } else {
            strcpy(node->expr_type, "int");
        }
    }
    eat(T_NUMBER);
}
```

Figura 5: Recursive descent logic in C

```
else if (current_token.type == T_STRING_VAL) {
    strcpy(node->expr_type, "string");
    eat(T_STRING_VAL);
}
else if (current_token.type == T_BOOL_VAL) {
    strcpy(node->expr_type, "bool");
    eat(T_BOOL_VAL);
}
else {
    printf("Parsing error...\n");
    exit(1);
}

strcpy(node->semi, current_token.lexeme);
eat(T_SEMI);

return node;
}
```

Figura 6: Recursive descent logic in C.

4.3 Syntax-Directed Translation (SDT) and Type Checking

After the AST is successfully built, the system needs to check if the code makes logical sense. This is called Semantic Analysis, while syntax checks if the sentence is written correctly, semantics checks if the meaning is correct. We do this using Syntax-Directed Translation (SDT).

Our python code reads the AST and compares the data types. For example, if you write `int x = 10;` the SDT sees that the Type `int` matches the number `10`, so it is a Success. however, if you write `string x = 10;` the syntax is fine, but the SDT detects that the types do not match and shows an “SDT error”, we also added a special validation rule: **if the user assigns an integer value to a float variable**, the system accepts it by doing an automatic implicit cast.

```
if ast["node_type"] == "VarDecl":
    declared_type = ast["var_type"]
    expression_type = ast["expr_type"]
    (variable) declared_type: Any
    if declared_type != expression_type:
        if not (declared_type == 'float' and expression_type == 'int'):
            sdt_verified = False
```

Figura 7: Semantic validation logic in python.

To summarize our semantic logic, the following table shows how our SDT module evaluates the rules during execution.

Production evaluated	Semantic action / rule applied	Final status
$T.type == E.type$	Direct assignment Types are identical	Success.
$T.Type == \text{float} \wedge E.type == \text{int}$	Implicit casting (Promotion from integer to float)	Success
$T.type \neq E.type$	Type mismatch error	SDT Error
Invalid Token Order	The input does not respect the grammar	Parsing Error

5 Results

5.1 TEST 1:

```

Parser & SDT - Grupo 5
Analizador Sintáctico y Semántico

int x = 10;

▶ Compilar Código

Parsing Success!
SDT Verified!

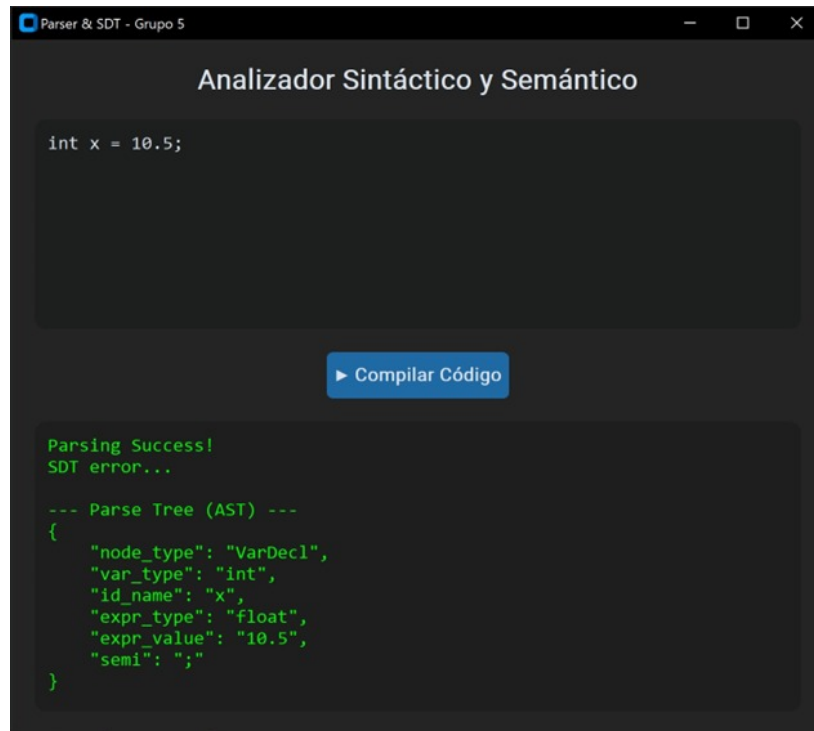
--- Parse Tree (AST) ---
{
  "node_type": "VarDecl",
  "var_type": "int",
  "id_name": "x",
  "expr_type": "int",
  "expr_value": "10",
  "semi": ";"
}

```

Figura 8: Output obtained from `int x = 10`.

This case was a success because the Syntax Analyzer correctly validated the input string against the defined grammar. The SDT successfully performed semantic verification, ensuring type consistency and generating an accurate Abstract Syntax Tree.

5.2 TEST 2:



```
Parser & SDT - Grupo 5

Analizador Sintáctico y Semántico

int x = 10.5;

▶ Compilar Código

Parsing Success!
SDT error...

--- Parse Tree (AST) ---
{
  "node_type": "VarDecl",
  "var_type": "int",
  "id_name": "x",
  "expr_type": "float",
  "expr_value": "10.5",
  "semi": ";"
}
```

Figura 9: Output obtained from `int x = 10.5`.

The success of this test case lies in the program's ability to identify a logical contradiction an attempt to assign a decimal value in this case 10.5 to a variable explicitly declared as an integer. While the sentence is structurally correct, the analyzer successfully detected that the data types are incompatible and blocked the process. This proves that the compiler's validation rules are working exactly as intended, protecting the program's integrity by rejecting operations that do not follow the established logic.

5.3 TEST 3:

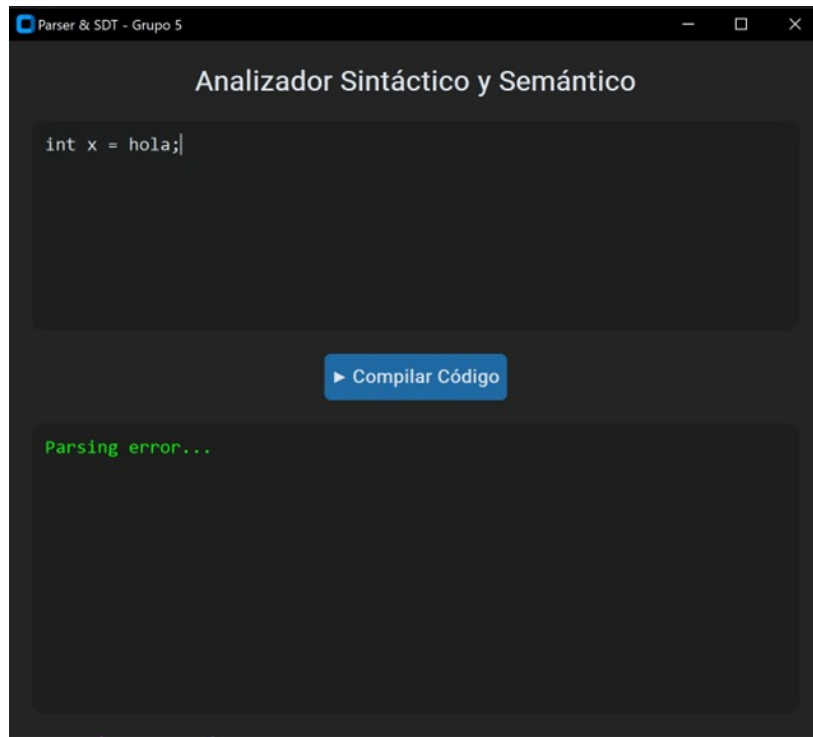
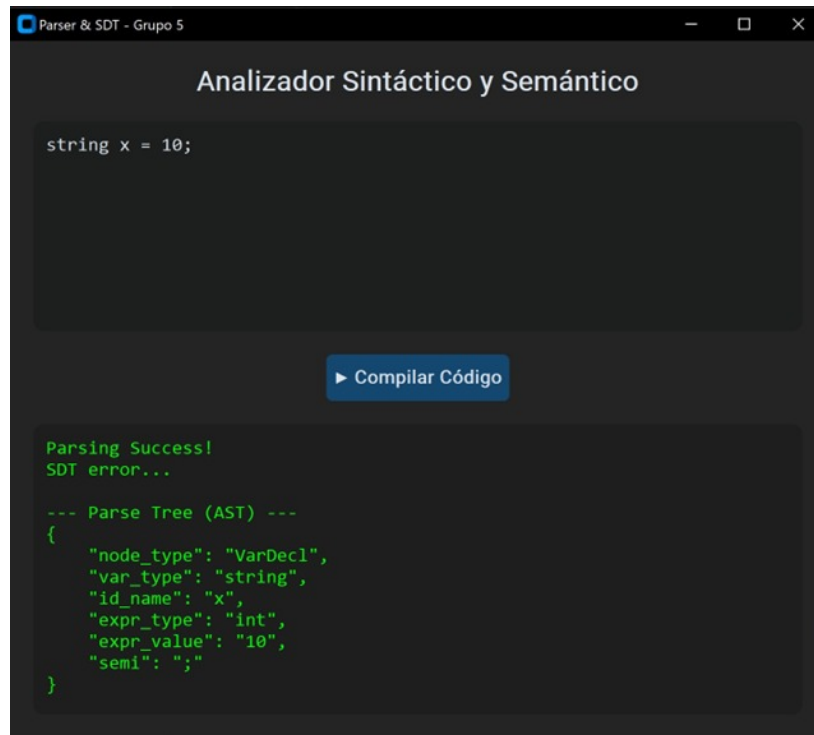


Figura 10: Output obtained from `int x = hola;`

The system correctly identified the input `int x = hola;` as an error because 'hola' is a string identifier being assigned to an integer variable. Since the grammar expects a numeric value to follow the integer declaration, the `hola` token caused an immediate breakdown in the expected structure. By triggering a Parsing Error, the analyzer proves it can distinguish between a properly formed statement and a sequence of tokens that does not follow the required syntax of the language.

5.4 TEST 4:



```
Parser & SDT - Grupo 5

Analizador Sintáctico y Semántico

string x = 10;

▶ Compilar Código

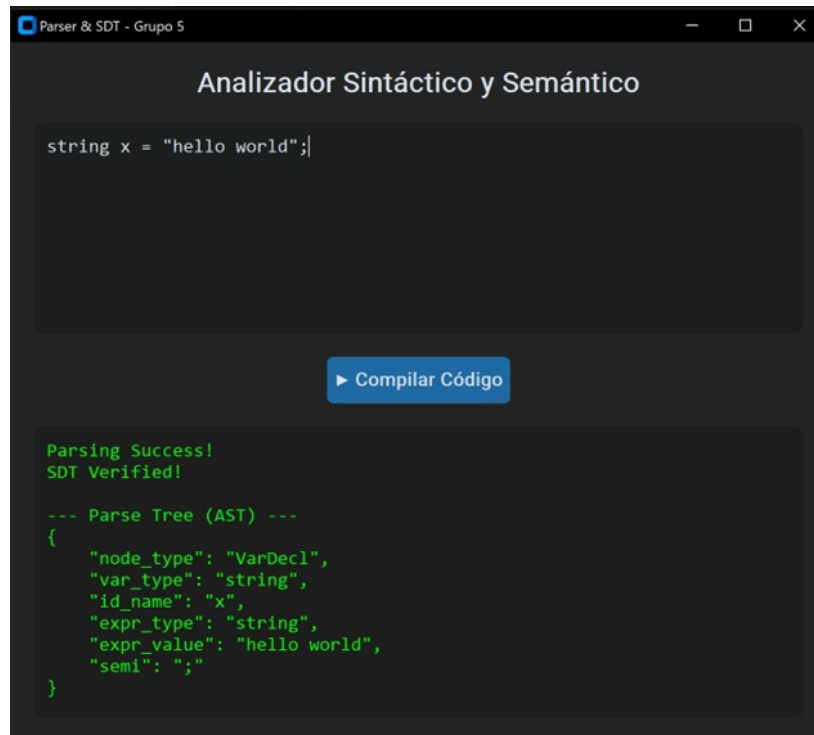
Parsing Success!
SDT error...

--- Parse Tree (AST) ---
{
  "node_type": "VarDecl",
  "var_type": "string",
  "id_name": "x",
  "expr_type": "int",
  "expr_value": "10",
  "semi": ";"
}
```

Figura 11: Output obtained from `string x = 10;`

The success of this test case lies in the program's ability to identify a logical contradiction: an attempt to assign a decimal value in this case 10 to a variable explicitly declared as a string. While the sentence is structurally correct, the analyzer successfully detected that the data types are incompatible and blocked the process. This proves that the compiler's validation rules are working exactly as intended, protecting the program's integrity by rejecting operations that do not follow the established logic.

5.5 TEST 5:



```
Parser & SDT - Grupo 5

Analizador Sintáctico y Semántico

string x = "hello world";

▶ Compilar Código

Parsing Success!
SDT Verified!

--- Parse Tree (AST) ---
{
  "node_type": "VarDecl",
  "var_type": "string",
  "id_name": "x",
  "expr_type": "string",
  "expr_value": "hello world",
  "semi": ";"
}
```

Figura 12: Output obtained from `string x = "hello world";`

This case was a success because the Syntax Analyzer correctly validated the input string against the defined grammar. The SDT successfully performed semantic verification, ensuring type consistency and generating an accurate Abstract Syntax Tree

6 Conclusions:

While developing this project, we initially expected it to be as challenging as our previous assignments, but we found the process to be much easier due to our modular approach. By splitting the workload into two distinct phases using C for structural parsing and Python for semantic logic we were able to simplify complex problems into manageable, functional stages.

The results of our testing demonstrate that this hybrid system is effective at maintaining code integrity. Our analyzer successfully distinguished between syntactic correctness and semantic logic. By correctly identifying and blocking errors like type mismatches such as attempting to assign a decimal value to an integer variable we proved that our Syntax Directed Translation module acts as a reliable gatekeeper for data consistency. Ultimately, this project illustrates how abstract concepts from formal grammar can be translated into a functioning computational system that ensures both structural and logical accuracy.

References

- [1] GeeksforGeeks, "Syntax Directed Translation in Compiler Design," GeeksforGeeks, mayo 08, 2023. [En línea]. Disponible en: <https://www.geeksforgeeks.org/compiler-design/syntax-directed-translation-in-compiler-design/>. [Accedido: abr. 22, 2026].
- [2] Arimetrics, "Analizador sintáctico - Parser," Glosario Digital Arimetrics, 2023. [En línea]. Disponible en: <https://www.arimetrics.com/glosario-digital/analizador-sintactico-parser>. [Accedido: 25-abr-2026].